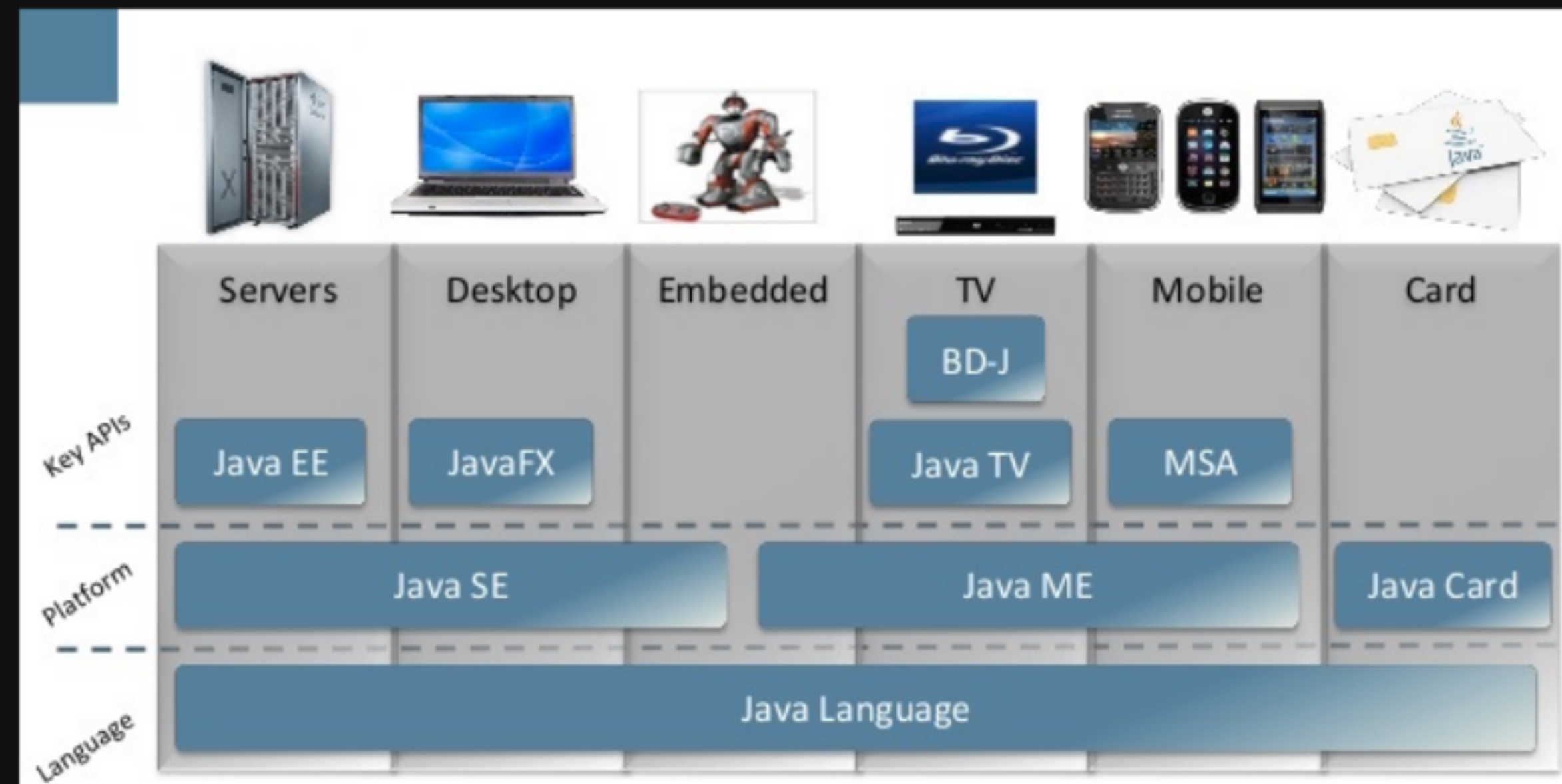


Enterprise Java

Created by <Academia de Código_>

Java Platforms

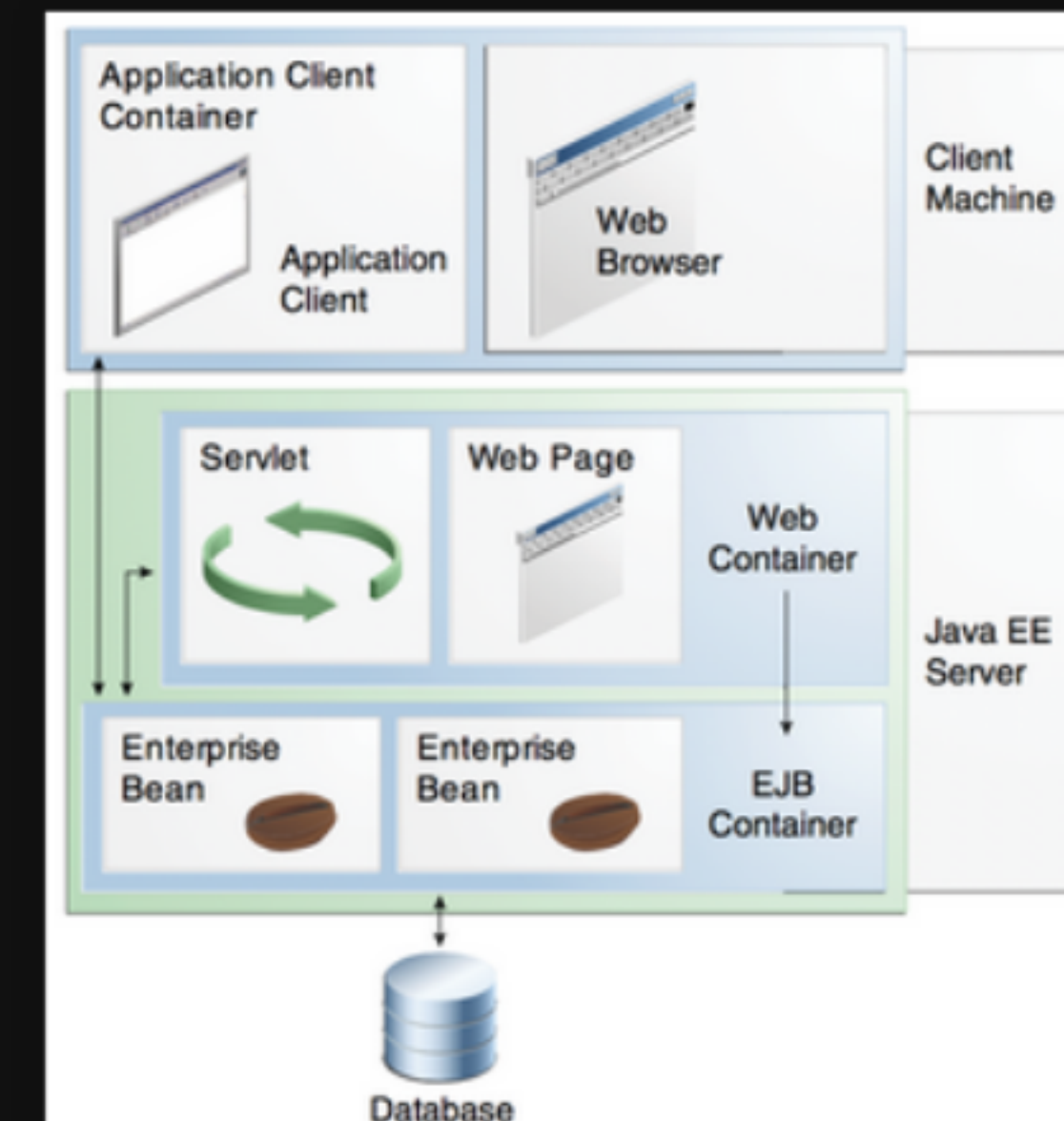


Java Enterprise Edition

The Java EE architecture makes enterprise applications easy to write because business logic is organized into reusable components

The Java EE server provides underlying services in the form of a container, avoiding handwriting many lines of code to handle transactions, state management, multithreading, resource pooling and other complex low-level details

- Java EE Server is the runtime portion of a Java EE product and provides EJB and Web containers
- EJB Container runs on the Java EE Server and manages the execution of Enterprise Java Beans
- Web Container runs on the Java EE Server and manages the execution of web pages, servlets, and some EJB components
- EJBs are Java components, containing business logic, that benefit from technical services provided by the container, usually in a declarative way, thanks to annotations

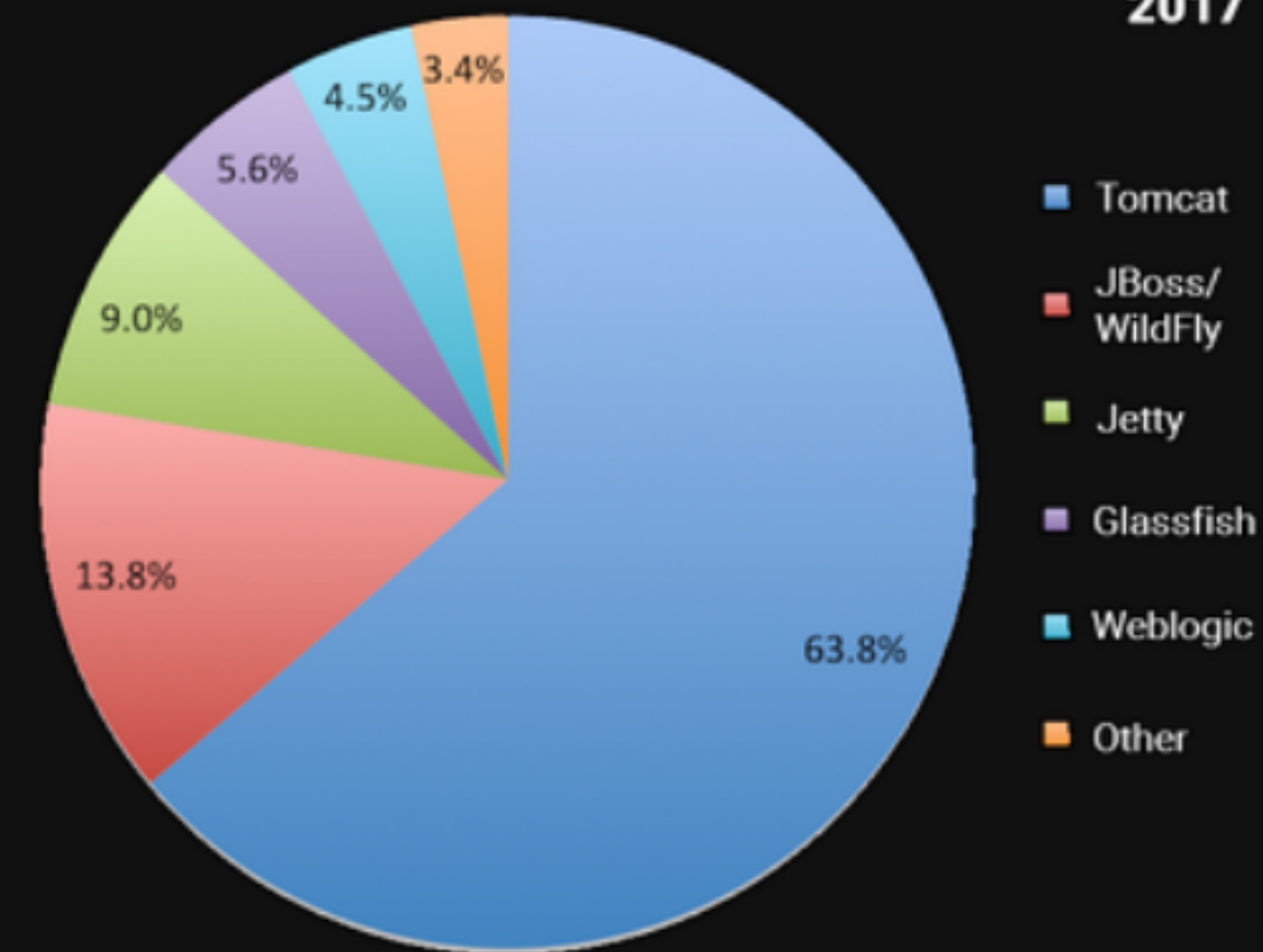


Container Usage in 2017

Tomcat and Jetty are just servlet containers, they implement only the servlets and jsp specification

Glassfish and JBoss are full Java EE servers

JAVA APPLICATION SERVER MARKET
2017



Glassfish is the Oracle JEE reference implementation

Spring vs Java EE

Both JavaEE and Spring work on the same core APIs (Servlet, JPA, JMS, etc), the difference is what glues things together, Spring or an AppServer

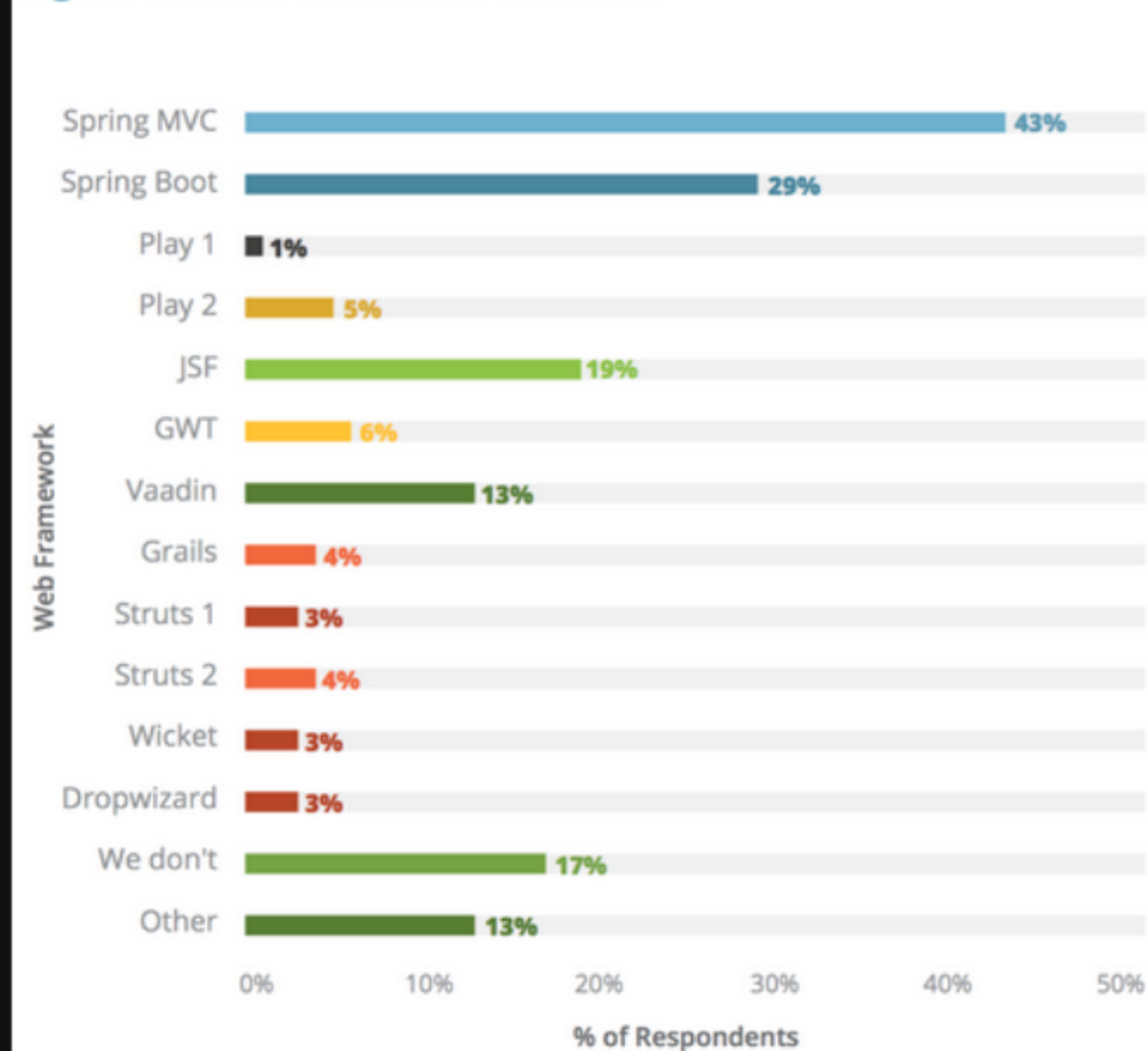
Even though there are some different APIs for things like dependency injection (Spring DI vs CDI), REST (JAX-RS vs SpringMVC) etc., they look and behave pretty similar to each other



Spring Usage in 2016

Spring is the most widely used web framework in the enterprise:

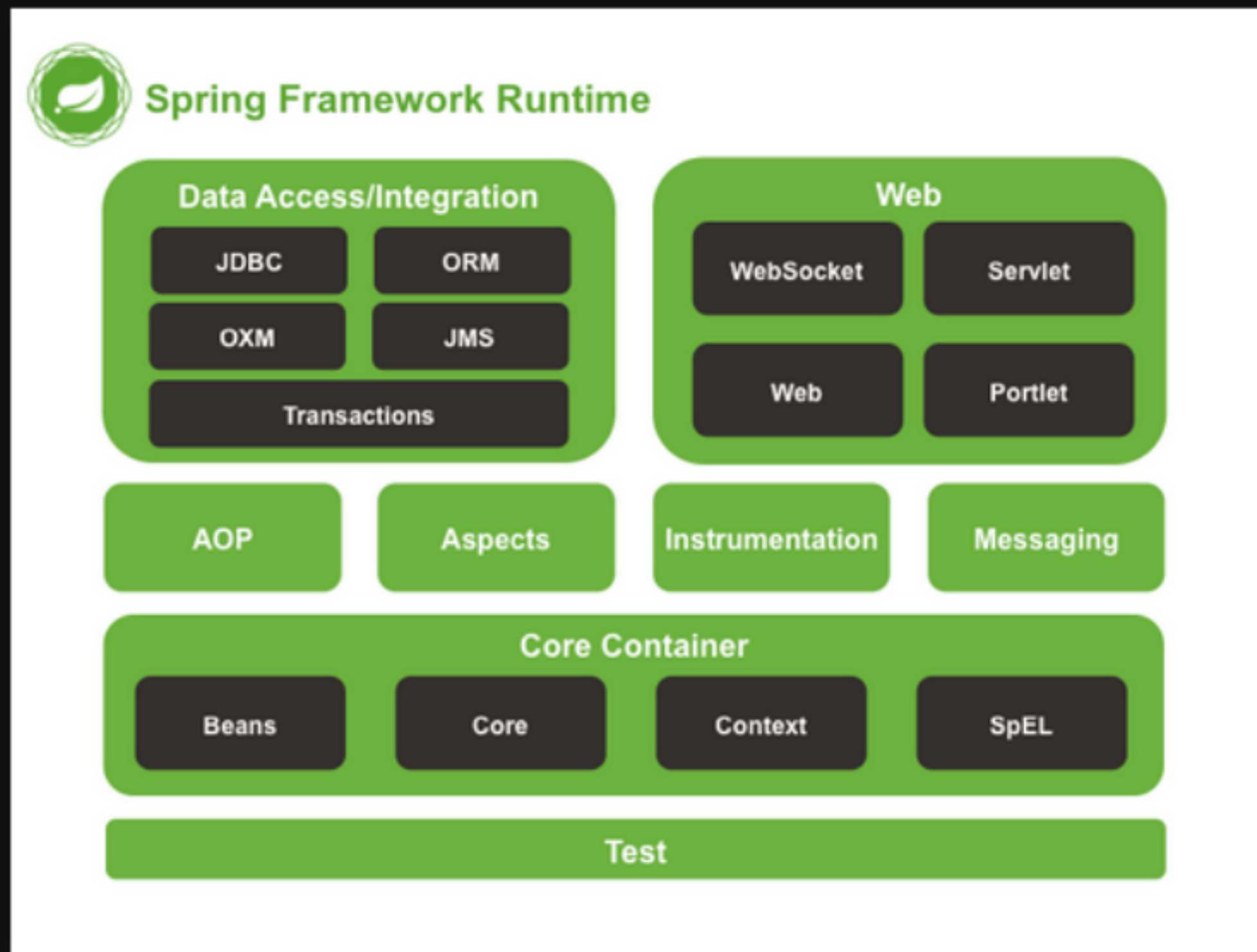
Figure 1.16 War of the web frameworks





Spring Framework

Consists of features organized into about 20 modules, grouped into the Core Container, AOP and Instrumentation, Messaging, Data Access/Integration, Web and Test:



Spring Core

Central part of the Spring Framework, upon which everything else is built on. Its core concepts are:

- Spring Beans - Contrary to EJB, Spring is designed to use POJOs
- Dependency Injection - Decreases object coupling, reduces boilerplate code and facilitates testing
- Inversion of Control - Management of software components and their injection into dependent objects
- Spring Context - Brings everything together, wiring up the whole environment
- Spring Expression Language - Concise language that provides great flexibility for configuring the Framework

Dependency Injection



A Java application consists of a number of objects that collaborate with each other. For this collaboration to take place, objects have dependencies on each other.

Dependency injection is a software design pattern that implements Inversion of Control for resolving dependencies

What this means is that a given class or system is no longer responsible for instantiating their own dependencies, but moves the responsibility for locating and attaching dependency objects to another class or a DI tool

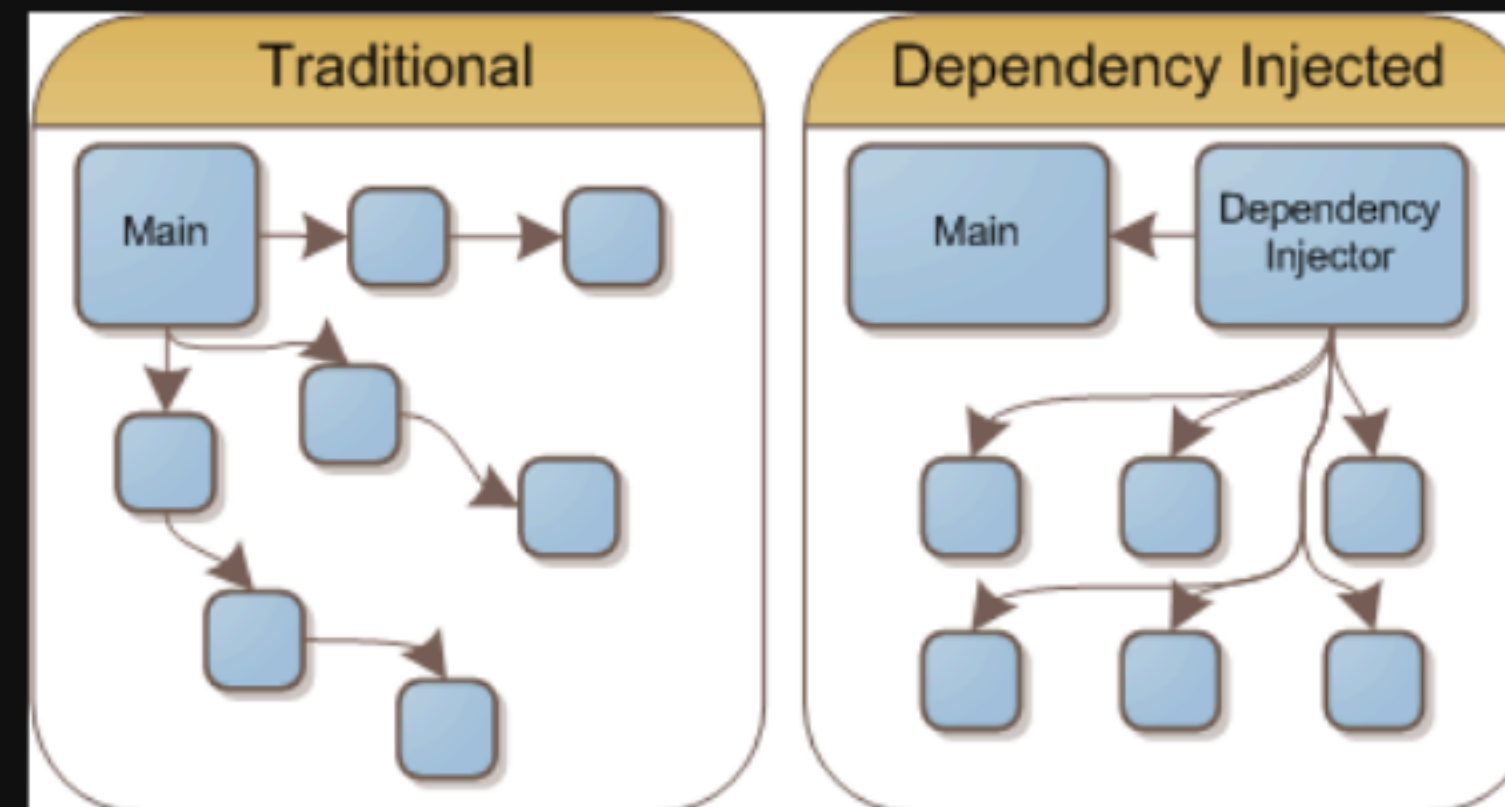
Benefits of DI



- decreases coupling between a class and its dependencies
- increases flexibility due to the class capability of working with different implementations of its dependencies
- increases testability by allowing easy injection of mocks or stubs
- increases code reuse, external configuration and injection of dependencies facilitates class usage in different contexts

IoC Container

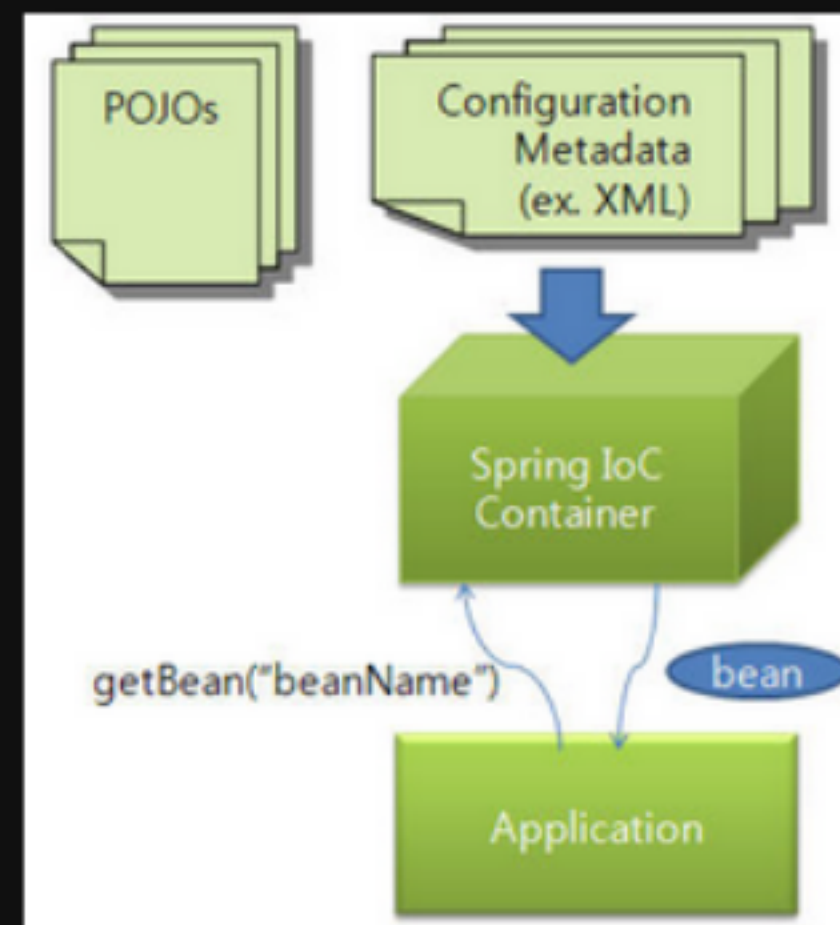
The Inversion of Control container manages object lifecycle and dependency injection



Spring Container

The most important part and the foundation of the Spring framework. Deals with configuration and lifecycle management of Spring Beans (POJOs)

The container is represented by the ApplicationContext interface and gets its instructions on what objects to instantiate, configure, and assemble by reading configuration metadata



Configuration Metadata

Tells the container how to instantiate, configure, and assemble the objects that compose the application

The following forms of metadata can be used with the Spring container:

- XML-based configuration
- Annotation-based configuration
- Java-based configuration

XML Metadata

```
<beans xmlns="...">

  <!-- external bean definitions go here -->
  <import resource="file.xml"/>

  <bean id="..." class="...">
    <!-- collaborators and configuration for this bean go here -->
  </bean>

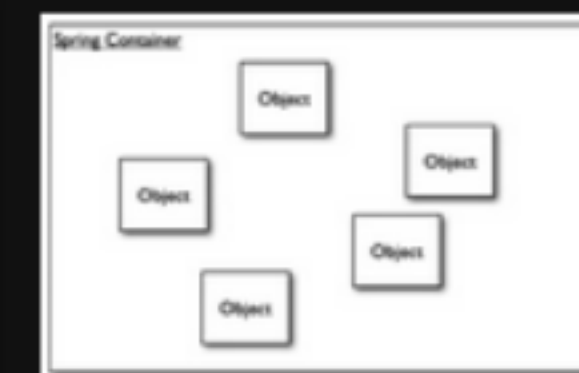
  <bean id="..." class="...">
    <!-- collaborators and configuration for this bean go here -->
  </bean>

  <!-- more bean definitions go here -->

</beans>
```

Container Instantiation

The container can load bean definitions from the classpath using ClassPathXmlApplicationContext



```
ApplicationContext context = new ClassPathXmlApplicationContext(  
    new String[] {"services.xml", "daos.xml"}  
);
```

Requesting Beans



```
// retrieve configured instance
UserService userService = context.getBean("userService", UserService.class);

// use configured instance
List<String> userList = userService.getUsernameList();
```

This example uses `getBean()` to retrieve bean instances, which couples this class with the Spring API. Ideally application code should have no dependency on Spring APIs at all, bean dependencies should be specified in the configuration metadata and the container will handle all the necessary wiring.

Spring Beans



Managed by the IoC Container and created according to their configuration metadata:

- class name name of the actual implementation class of the bean being defined (package-qualified)
- configuration settings for the newly created object, for example, the number of connections to use in a bean that manages a connection pool, or the size limit of the pool
- references to other beans that are needed for the bean to do its work (collaborators or dependencies)
- behavioral configuration elements state how the bean should behave in the container (scope, lifecycle callbacks, etc.)

Bean Instantiation

Most Spring users prefer to work with JavaBeans with only a default (no-argument) constructor and appropriate setters and getters , but the Spring IoC container can manage virtually any class, even more exotic non-bean-style classes

```
// Bean definition  
<bean id="hello" class="org.academiadecodigo.spring.HelloWorld"/>
```

```
// Bean instantiation using a constructor with no arguments  
HelloWorld h = context.getBean("hello", HelloWorld.class);  
  
h.sayHello();
```

with Static Factory Method

Given the following code:

```
public class ClientService {  
    private static ClientService clientService = new ClientService();  
    private ClientService() {}  
  
    public static ClientService getInstance() {  
        return clientService;  
    }  
}
```

The bean definition for ClientService is:

```
<bean id="clientService" class="examples.ClientService" factory-method="getInstance"/>
```

with Instance Factory Method

Given the following code:

```
public class DefaultServiceLocator {  
  
    private static ClientService clientService = new ClientServiceImpl();  
    public DefaultServiceLocator() {}  
  
    public ClientService createClientServiceInstance() {  
        return clientService;  
    }  
}
```

The bean definition for ClientService is:

```
<!-- the factory bean, which contains a method called createClientServiceInstance() -->  
<bean id="serviceLocator" class="examples.DefaultServiceLocator"/>  
  
<!-- the bean to be created via the factory bean -->  
<bean id="clientService" factory-bean="serviceLocator"  
    factory-method="createClientServiceInstance"/>
```

Bean Dependencies

Even the simplest application has a few objects that work together to achieve the desired functionality. Dependency injection (DI) is a process whereby objects define their dependencies only through:

- constructor arguments
- arguments to a factory method
- properties set on the object instance after it is instantiated

The container then injects those dependencies when it creates the bean using one of:

- Constructor-based dependency injection
- Setter-based dependency injection

Constructor Based DI

```
public class SimpleMovieLister {  
  
    // the SimpleMovieLister has a dependency on a MovieFinder  
    private MovieFinder movieFinder;  
  
    // a constructor so that the Spring container can inject a MovieFinder  
    public SimpleMovieLister(MovieFinder movieFinder) {  
        this.movieFinder = movieFinder;  
    }  
  
    // business logic that actually uses the injected MovieFinder  
  
}
```

We could wire the above bean dependencies using:

```
<bean id="movieFinder" class="MovieFinder"/>  
  
<bean id="simpleMovieLister" class="SimpleMovieLister">  
    <constructor-arg name="movieFinder" ref="movieFinder"/>  
</bean>
```

Setter Based DI

```
public class SimpleMovieLister {  
  
    // the SimpleMovieLister has a dependency on the MovieFinder  
    private MovieFinder movieFinder;  
  
    // a setter method so that the Spring container can inject a MovieFinder  
    public void setMovieFinder(MovieFinder movieFinder) {  
        this.movieFinder = movieFinder;  
    }  
  
    // business logic that actually uses the injected MovieFinder  
  
}
```

We could wire the above bean dependencies using:

```
<bean id="movieFinder" class="MovieFinder"/>  
  
<bean id="simpleMovieLister" class="SimpleMovieLister">  
    <property name="movieFinder" ref="movieFinder"/>  
</bean>
```

Constructor-based or Setter-based DI?

Since you can mix constructor-based and setter-based DI, it is a good rule of thumb to use constructors for mandatory dependencies and setter methods for optional dependencies

Bean Configuration

Besides dependent objects, Spring can also inject configuration properties using the value attribute of the property element

```
<!-- Bootcamp data source using C3P0 -->
<bean id="bootcampDB" class="com.mchange.v2.c3p0.ComboPooledDataSource">
  <property name="driverClass" value="com.mysql.cj.jdbc.Driver" />
  <property name="jdbcUrl" value="jdbc:mysql://localhost:3306/bootcamp" />
  <property name="user" value="root" />
  <property name="password" value="root" />
  <property name="maxPoolSize" value="50" />
  <property name="minPoolSize" value="10" />
  <property name="maxStatements" value="100" />
  <property name="testConnectionOnCheckout" value="true" />
</bean>
```

Values are defined as string representations, Spring will convert to the actual type of the property or argument

{ Hands On }

Spring Hello World

Collection Injection

Besides references and values, Spring can also inject collection dependencies using the following elements:

Tag	Inner Tag	Java Type	Specification
<list>	<value>	java.util.List<E>	Allows duplicate entries
<set>	<value>	java.util.Set<E>	Does not allow duplicate entries
<map>	<entry>	java.util.Map<K,V>	Key-Value pair of any object type
<props>	<prop>	java.util.Properties	Key-Value pair of type String

Collection Injection

```
public class Person {  
    private Map<String, Float> accounts;  
    public void setAccounts(Map<String, Float> accounts) {  
        this.accounts = accounts;  
    }  
}
```

```
<beans>  
    <bean id="person" class="Person">  
        <property name="accounts">  
            <map>  
                <entry key="one" value="9.99"/>  
                <entry key="two" value="2.75"/>  
                <entry key="six" value="3.99"/>  
            </map>  
        </property>  
    </bean>  
</beans>
```

Bean Scope

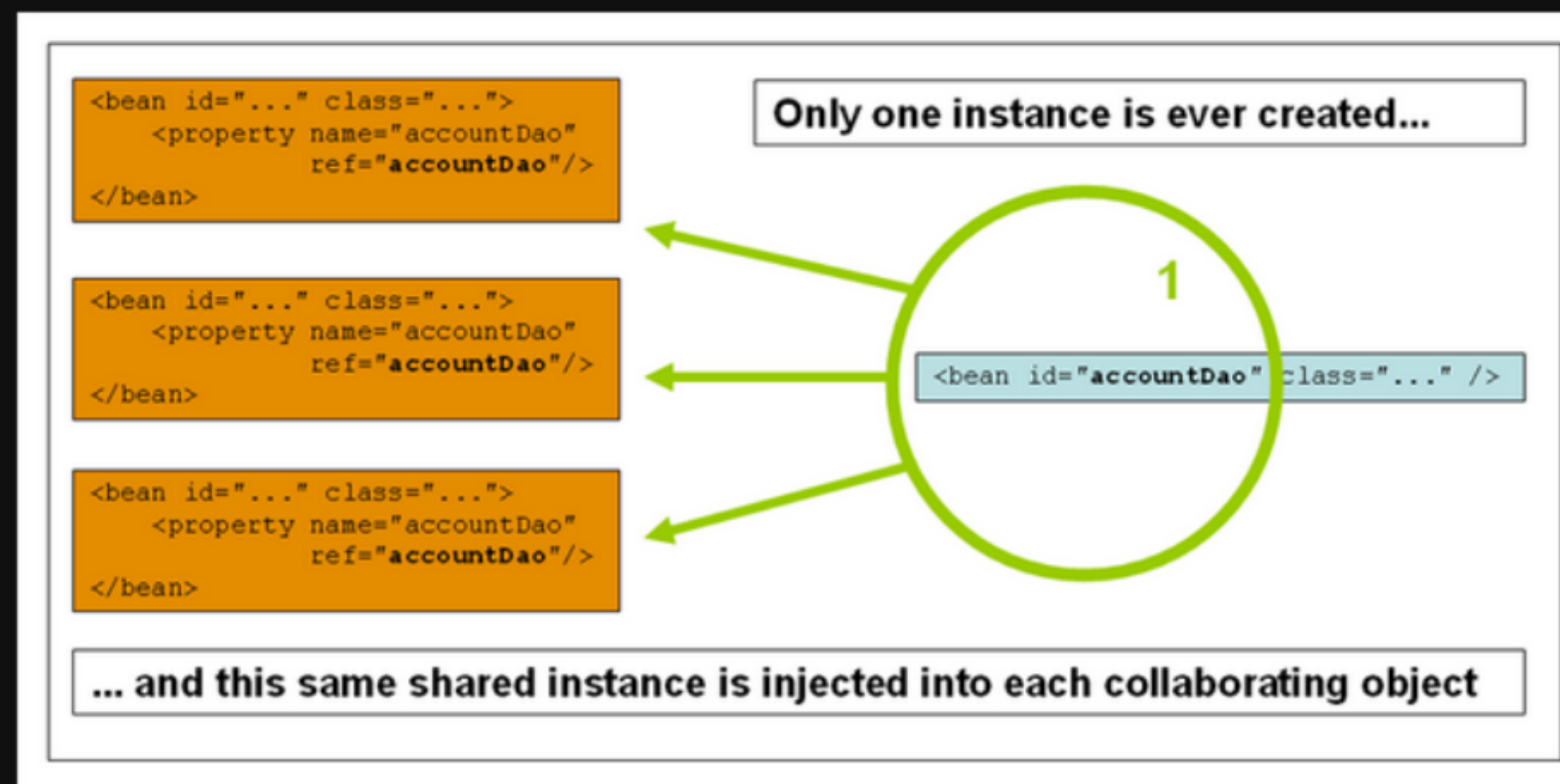
Spring Beans have a scope property that defines how many instances of the bean can Spring instantiate

For now, we will use two different bean scopes:

- singleton is the default scope and restricts spring to one single object instance per IoC container
- prototype allows spring to create any number of object instances

Singleton Scope

All requests for beans with an id matching the bean definition result in that one specific bean instance being returned by the container

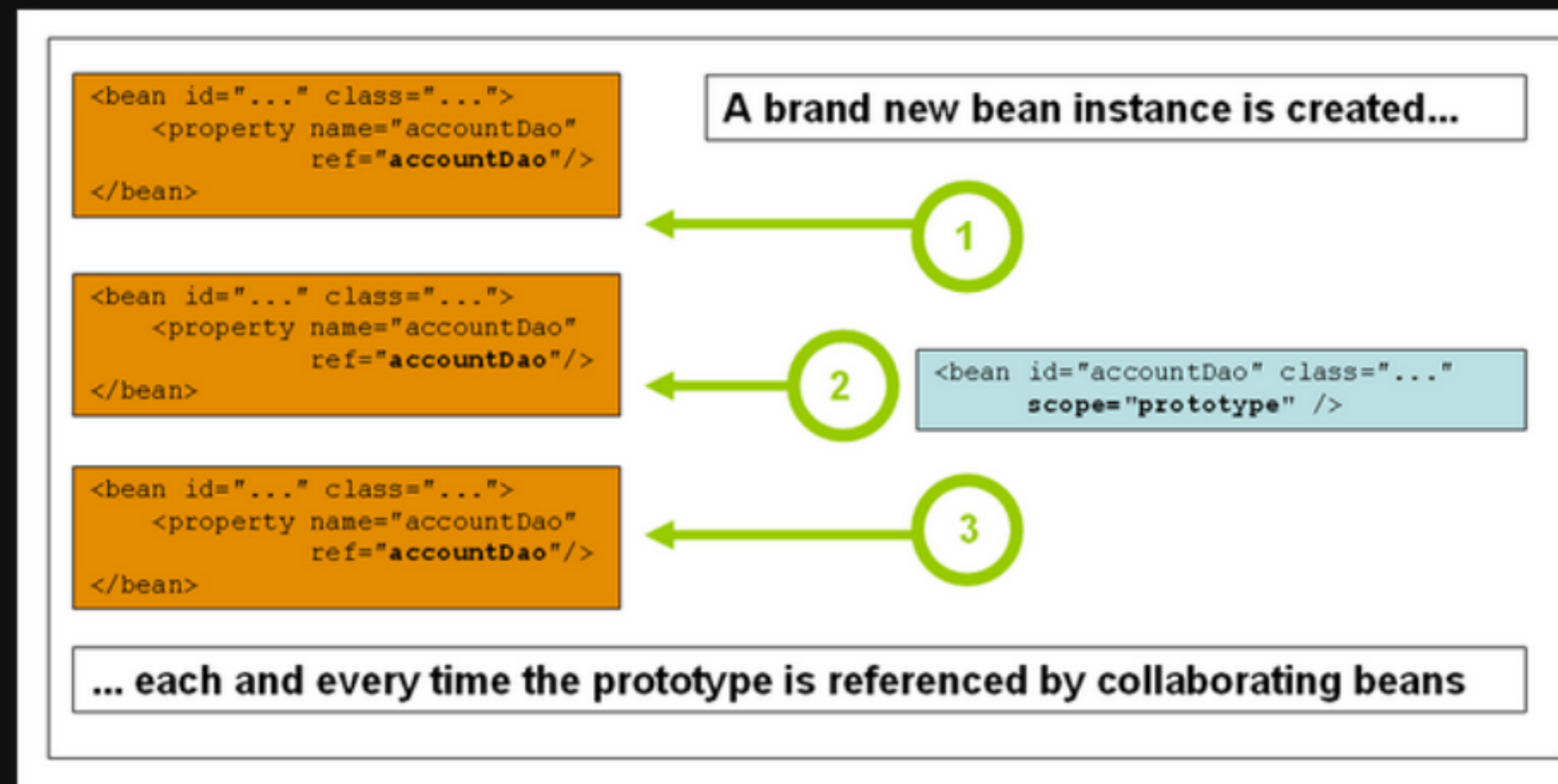


```
<bean id="accountService" class="com.foo.DefaultAccountService"/>

<!-- the following is equivalent (singleton scope is the default) -->
<bean id="accountService" class="com.foo.DefaultAccountService" scope="singleton"/>
```

Prototype Scope

All requests for beans with an id matching the bean definition result in the creation of a new bean instance every time a request for that specific bean is made



```
<bean id="accountService" class="com.foo.DefaultAccountService" scope="prototype"/>
```

Singleton vs Prototype

Which one should I use?

As a rule of thumb, use the prototype scope for all stateful beans
and the singleton scope for stateless beans

Lazy Initialization

By default, Spring container eagerly creates and configures all singleton beans as part of the initialization process

This default behaviour discovers errors in the configuration or environment immediately, as opposed to hours or even days later

When this behavior is not desirable, pre-instantiation of a singleton bean can be prevented by marking the bean definition as lazy-initialized

```
<bean id="lazy" class="ExpensiveToCreateBean" lazy-init="true"/>  
<bean id="not.lazy" class="SimpleBean"/>
```

Bean Lifecycle

Spring beans can receive lifecycle callbacks through the implementation of the InitializingBean and DisposableBean interfaces or alternatively, to avoid coupling objects with Spring APIs, through the bean configuration metadata

org.springframework.beans.factory.InitializingBean

void afterPropertiesSet () throws Exception;

```
public class MyBean implements
InitializingBean {
    public void afterPropertiesSet () {
        // Initialization work
    }
}
```

```
<bean id="myBean"
      class="MyBean" init-method="init"/>
```

```
public class MyBean {
    public void init () {
        // Initialization work
    }
}
```

org.springframework.beans.factory.DisposableBean

void destroy () throws Exception;

```
public class MyBean implements
DisposableBean {
    public void destroy () {
        // Destruction work
    }
}
```

```
<bean id="myBean"
      class="MyBean" destroy-method="des"/>
```

```
public class MyBean {
    public void des () {
        // Destruction work
    }
}
```

Maven Configuration

Add the following dependencies to your pom.

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context</artifactId>
  <version>4.3.11.RELEASE</version>
</dependency>

<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-orm</artifactId>
  <version>4.3.11.RELEASE</version>
</dependency>

<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-c3p0</artifactId>
  <version>5.3.6.Final</version>
</dependency>
```

Metadata

```
<bean class="org.springframework.orm.jpa.support.PersistenceAnnotationBeanPostProcessor"/>

<!-- c3p0 database connection pool management -->
<bean id="dataSource" class="com.mchange.v2.c3p0.ComboPooledDataSource" destroy-method="close">
  <property name="driverClass" value="com.mysql.cj.jdbc.Driver"/>
  <property name="jdbcUrl" value="jdbc:mysql://localhost:3306/ac"/>
  <property name="user" value="root"/>

  <!-- c3p0 database connection pool management properties -->
  <property name="minPoolSize" value="1"/>
  <property name="maxPoolSize" value="10"/>
  <property name="acquireIncrement" value="1"/>
  <property name="maxStatements" value="0"/>

  <!-- destroy idle pool after 300ms -->
  <property name="idleConnectionTestPeriod" value="300"/>

  <!-- destroy db connection if it fails to login after 7200ms -->
  <property name="loginTimeout" value="7200"/>
</bean>

<!-- spring jpa entity manager factory configuration -->
<bean id="emFactory" class="org.springframework.orm.jpa.LocalContainerEntityManagerFactoryBean">
  <property name="dataSource" ref="dataSource"/>
  <property name="packagesToScan" value="org.academiadecodigo.bootcamp.model"/>
</bean>
```

An EntityManagerFactory can now be injected in the application using the [@PersistenceUnit](#) annotation

Metadata

```
<property name="maxStatements" value="0"/>

<!-- destroy idle pool after 300ms -->
<property name="idleConnectionTestPeriod" value="300"/>

<!-- destroy db connection if it fails to login after 7200ms -->
<property name="loginTimeout" value="7200"/>

</bean>

<!-- spring jpa entity manager factory configuration -->
<bean id="emFactory" class="org.springframework.orm.jpa.LocalContainerEntityManagerFactoryBean">
  <property name="dataSource" ref="dataSource"/>
  <property name="packagesToScan" value="org.academiadecodigo.bootcamp.model"/>
  <property name="jpaVendorAdapter">
    <bean class="org.springframework.orm.jpa.vendor.HibernateJpaVendorAdapter"/>
  </property>
  <property name="jpaProperties">
    <map>
      <entry key="hibernate.dialect" value="org.hibernate.dialect.MySQL8Dialect"/>
      <entry key="hibernate.show_sql" value="true" />
      <!-- WARNING: Will re-create the database schema, causing data loss -->
      <entry key="hibernate.hbm2ddl.auto" value="create"/>
    </map>
  </property>
</bean>
```

An EntityManagerFactory can now be injected in the application using the `@PersistenceUnit` annotation

{ Exercise }

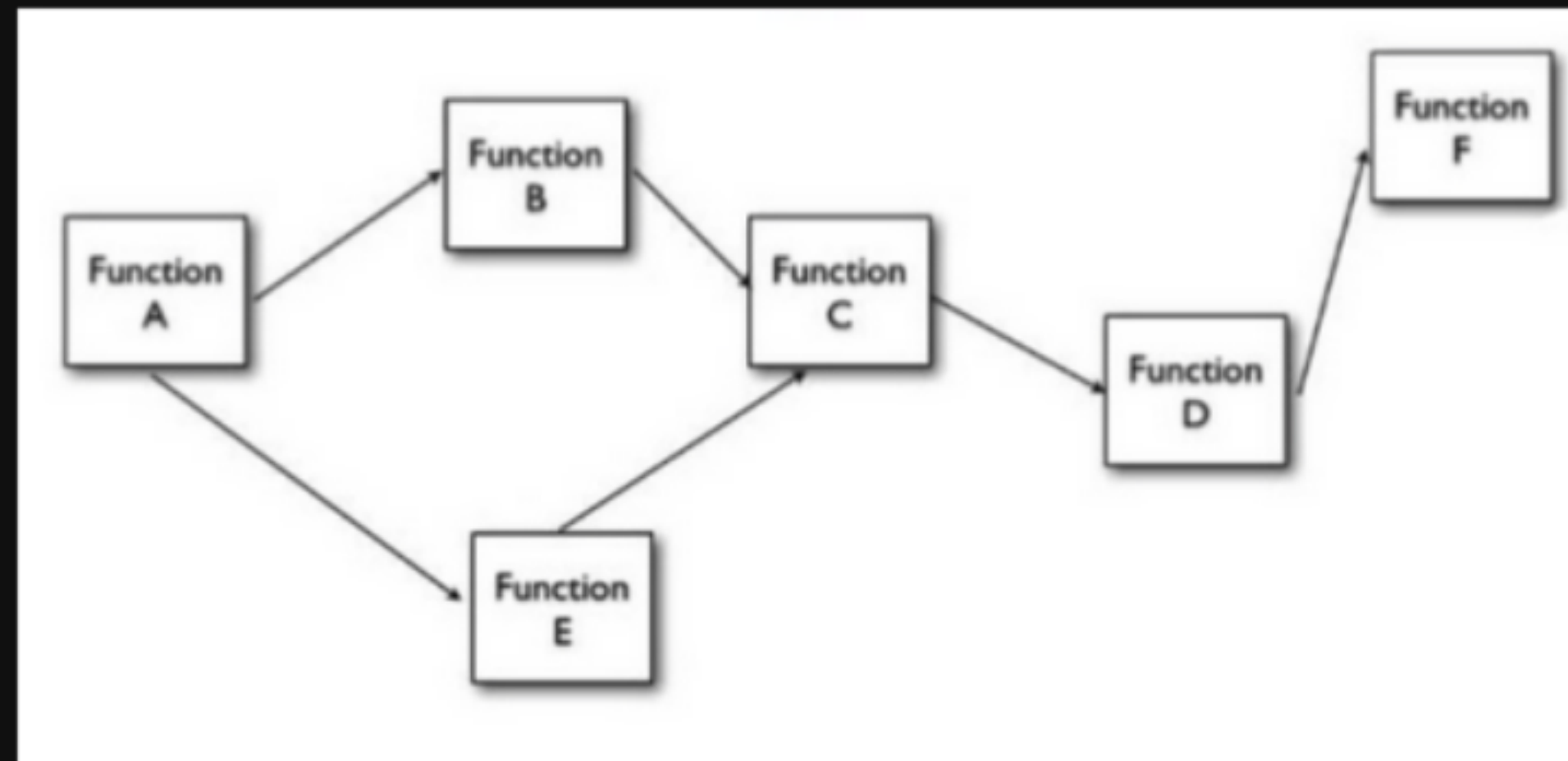
Integrate the Spring framework with the JavaBank application

Aspect Oriented Programming

...but just the basics...

In the beginning...

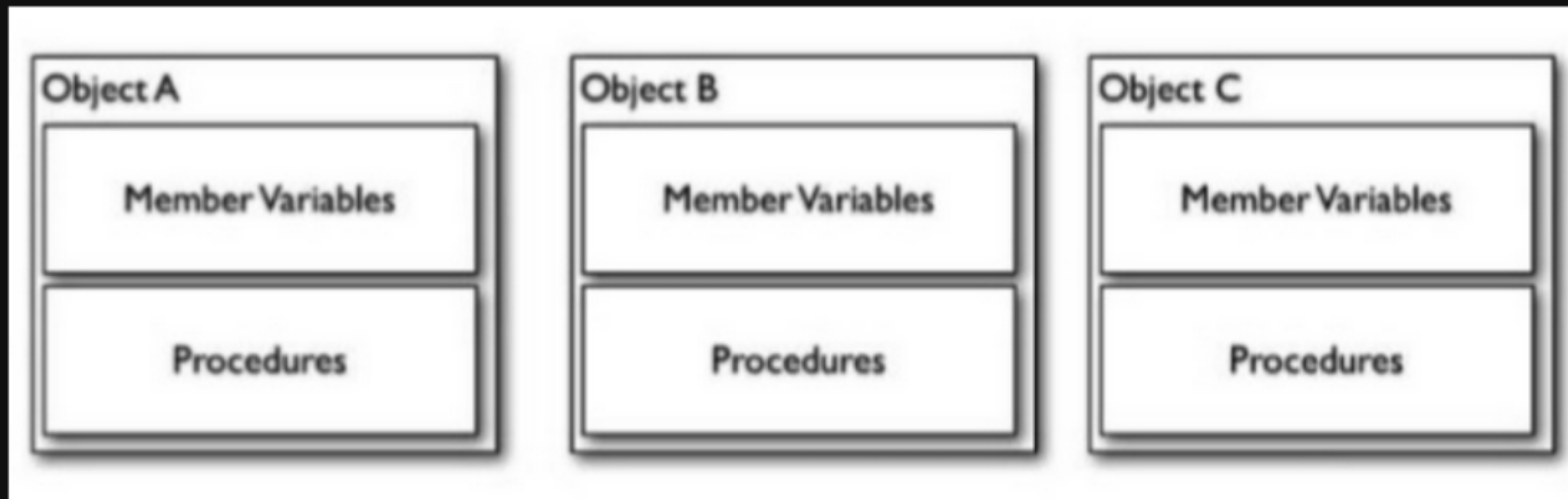
... we had Procedural Programming



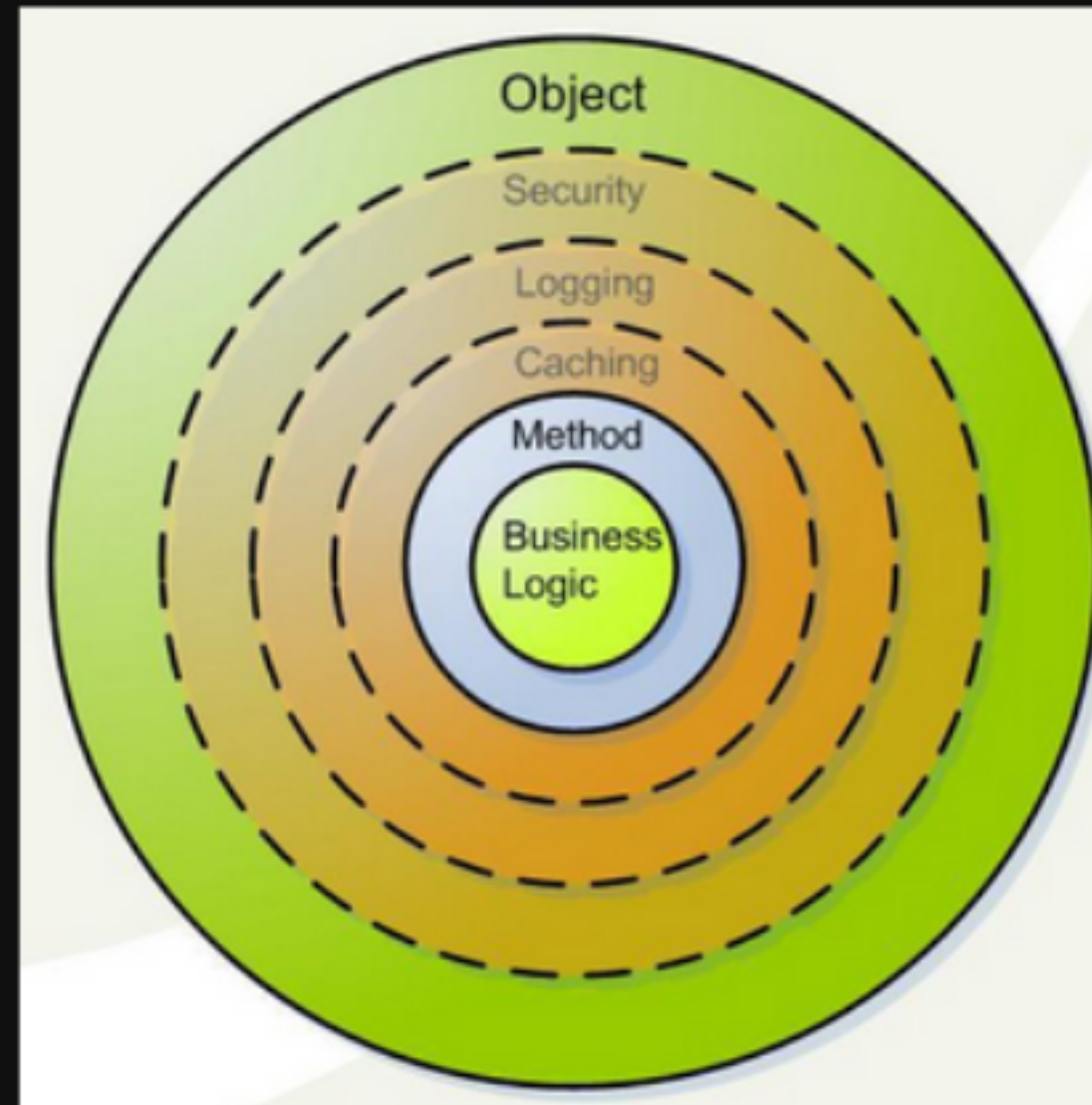
...later we evolved...



... into Object Oriented Programming

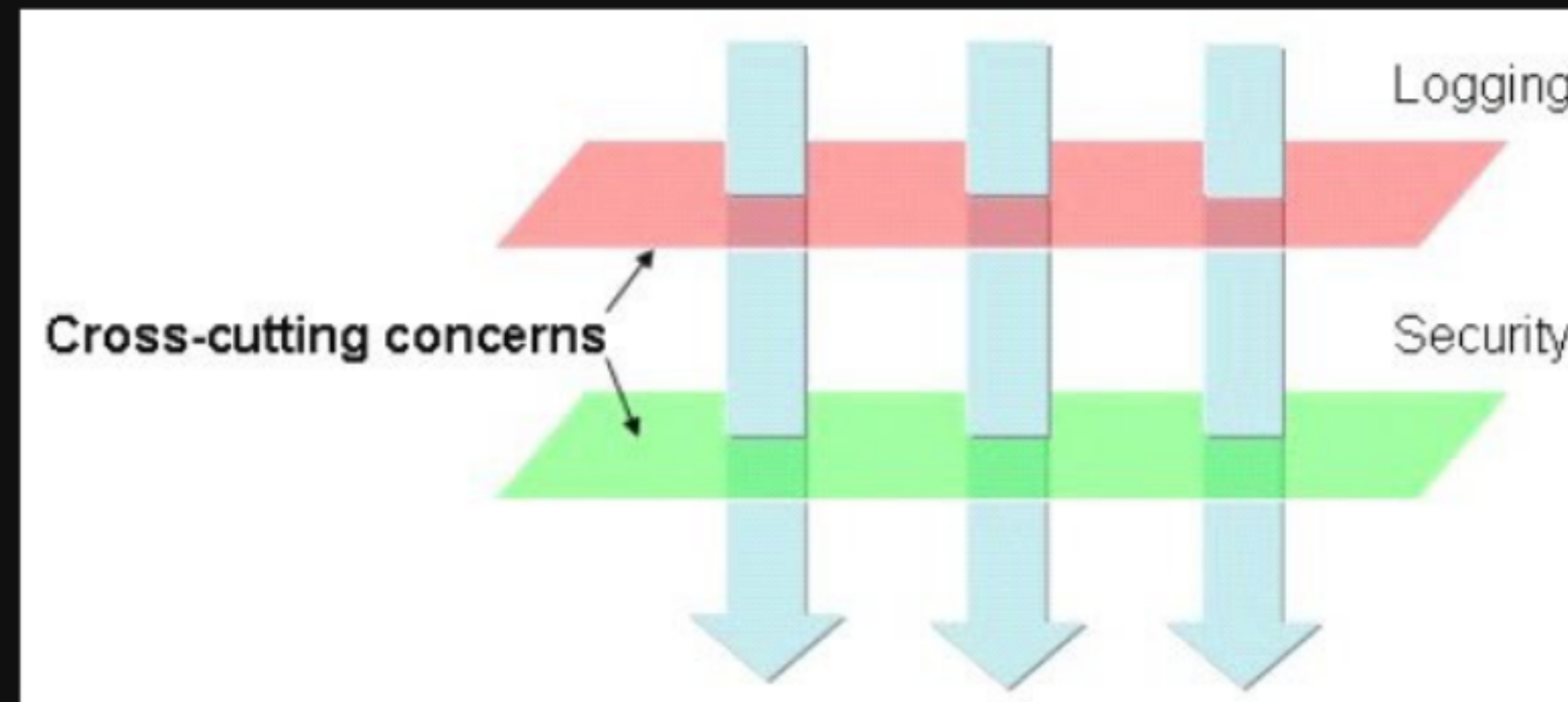


...and now we have AOP



What is AOP?

- Software Programming Paradigm
- Extends Object Oriented Programming
- Enables Modularization of Cross-cutting Concerns



What problem does AOP solve?

Let's examine the withdraw method for a banking application

Business Logic

Business

```
public void withdraw(long accId, int amount) {  
    Account account = (Account)session.get(Account.class, accId);  
    double balance = account.getBalance();  
    balance = balance - amount;  
    account.setBalance(balance);  
    session.save(balance);  
}
```

Authorization

Business

Permission

```
public void withdraw(long accId, int amount) {  
    if (hasPermission(accId)) {  
        Account account = (Account)session.get(Account.class, accId);  
        double balance = account.getBalance();  
        balance = balance - amount;  
        account.setBalance(balance);  
        session.save(balance);  
    } else {  
        throw new SecurityException("Access Denied");  
    }  
}
```

Logging

```
public void withdraw(long accId, int amount) {  
    if (hasPermission(accId)) {  
        bankLogger.info(accId + " withdraw amount " + amount);  
  
        Account account = (Account)session.get(Account.class, accId);  
        double balance = account.getBalance();  
        balance = balance - amount;  
        account.setBalance(balance);  
        session.save(balance);  
    } else {  
        throw new SecurityException("Access Denied");  
    }  
}
```

Business

Permission

Logger

Transaction Management

```
public void withdraw(long accId, int amount) {  
    if (hasPermission(accId)) {  
        bankLogger.info(accId + " withdraw amount " + amount);  
  
        Transaction tx = null;  
        try {  
            Transaction tx = session.beginTransaction();  
  
            Account account = (Account)session.get(Account.class, accId);  
            double balance = account.getBalance();  
            balance = balance - amount;  
            account.setBalance(balance);  
            session.save(account);  
  
            tx.commit();  
        } catch (RuntimeException ex) {  
            if (tx != null) tx.rollback();  
        }  
    } else {  
        throw new SecurityException("Access Denied");  
    }  
}
```

Business

Permission

Logger

Transaction

Service Layer

Service A

```
public void withdraw(long accId, int amount) {
    if (hasPermission(accId)) {
        bankLogger.info(accId + " withdraw amount " + amount);

        Transaction tx = null;
        try {
            Transaction tx = session.beginTransaction();

            Account account = (Account)session.get(Account.class, accId);
            double balance = account.getBalance();
            balance = balance - amount;
            account.setBalance(balance);
            session.save(balance);

            tx.commit();
        } catch (RuntimeException ex) {
            if (tx != null) tx.rollback();
        }
    } else {
        throw new SecurityException("Access Denied");
    }
}
```

Business

Permission

Logger

Transaction

Service B

```
public void withdraw(long accId, int amount) {
    if (hasPermission(accId)) {
        bankLogger.info(accId + " withdraw amount " + amount);

        Transaction tx = null;
        try {
            Transaction tx = session.beginTransaction();

            Account account = (Account)session.get(Account.class, accId);
            double balance = account.getBalance();
            balance = balance - amount;
            account.setBalance(balance);
            session.save(balance);

            tx.commit();
        } catch (RuntimeException ex) {
            if (tx != null) tx.rollback();
        }
    } else {
        throw new SecurityException("Access Denied");
    }
}
```

Service C

```
public void withdraw(long accId, int amount) {
    if (hasPermission(accId)) {
        bankLogger.info(accId + " withdraw amount " + amount);

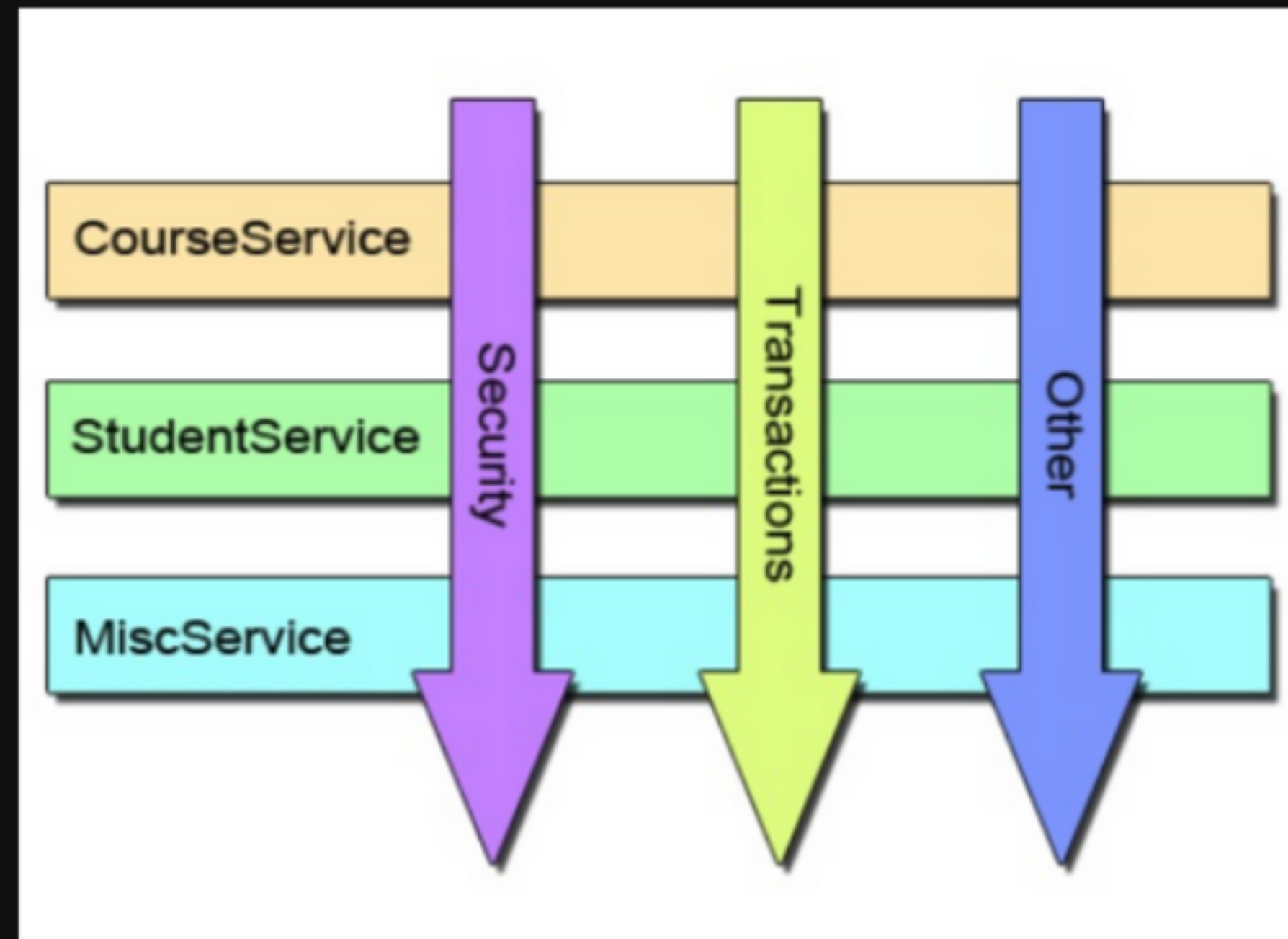
        Transaction tx = null;
        try {
            Transaction tx = session.beginTransaction();

            Account account = (Account)session.get(Account.class, accId);
            double balance = account.getBalance();
            balance = balance - amount;
            account.setBalance(balance);
            session.save(balance);

            tx.commit();
        } catch (RuntimeException ex) {
            if (tx != null) tx.rollback();
        }
    } else {
        throw new SecurityException("Access Denied");
    }
}
```

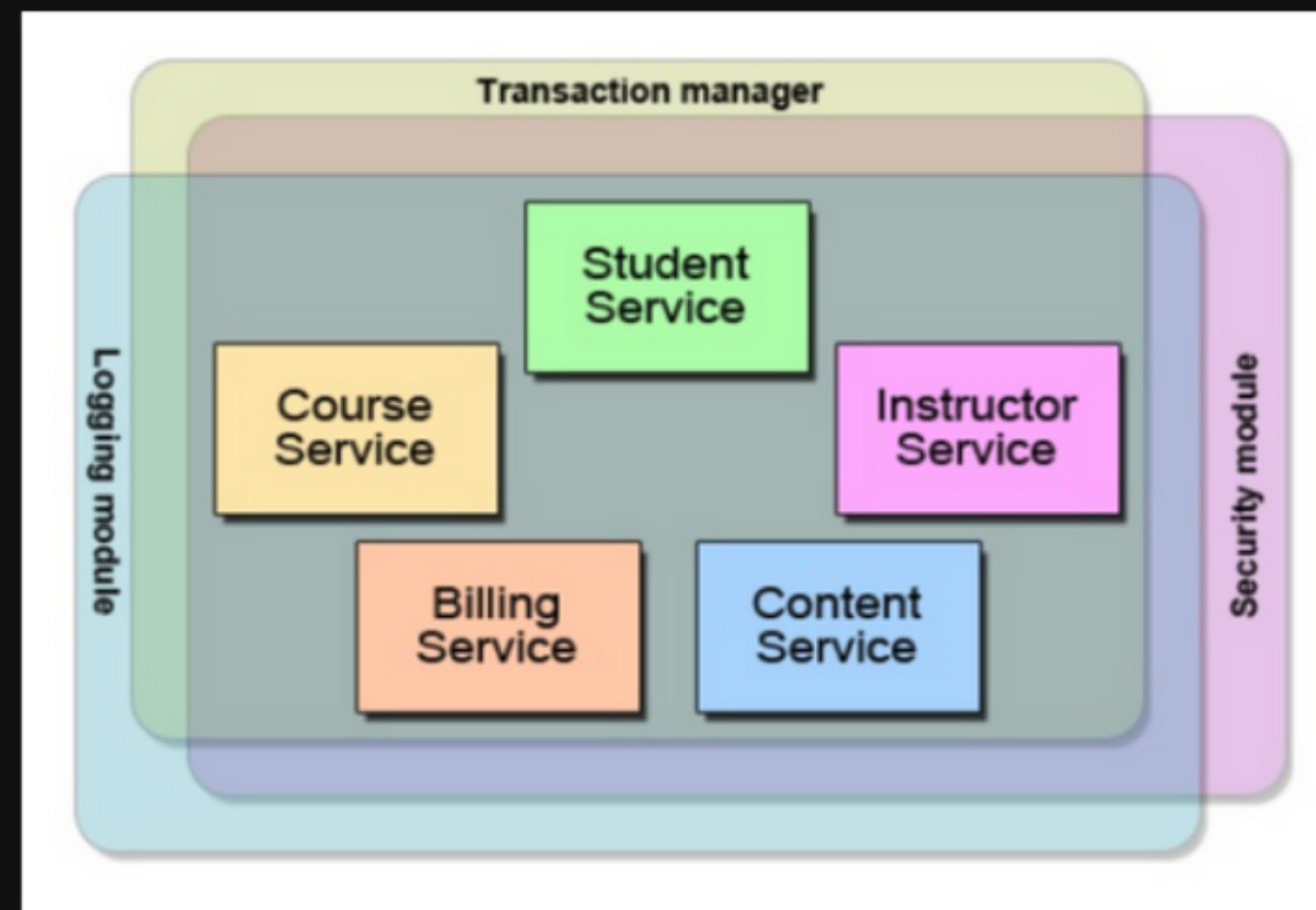
So... What problem does AOP solve?

- Addresses cross-cutting concerns
- Separates business logic
- Promotes reusability of code

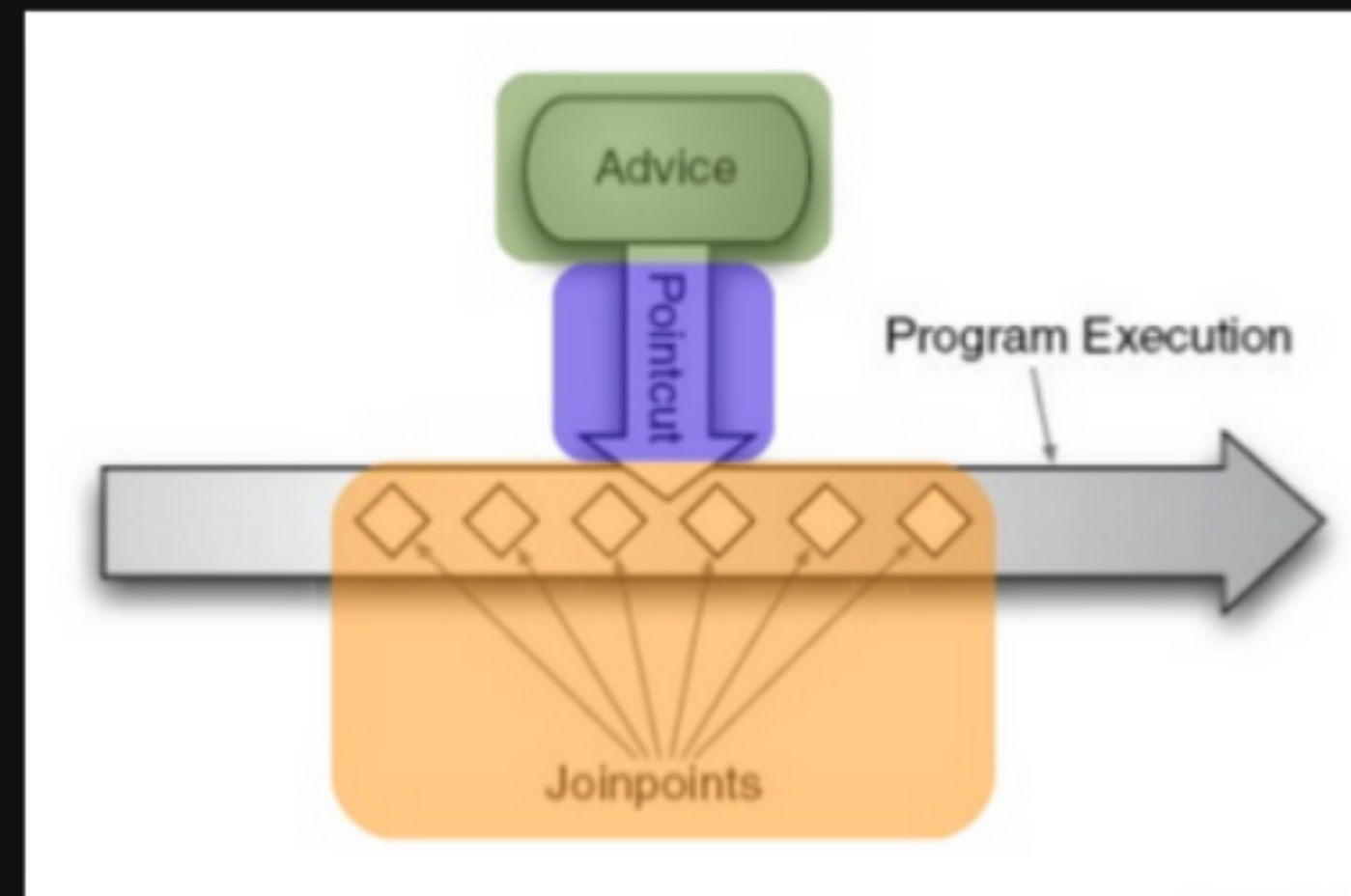


Aspects

An aspect is the cross-cutting functionality that will be modularized in AOP, such as logging, validation, caching, security, transactions, monitoring, error handling, etc.



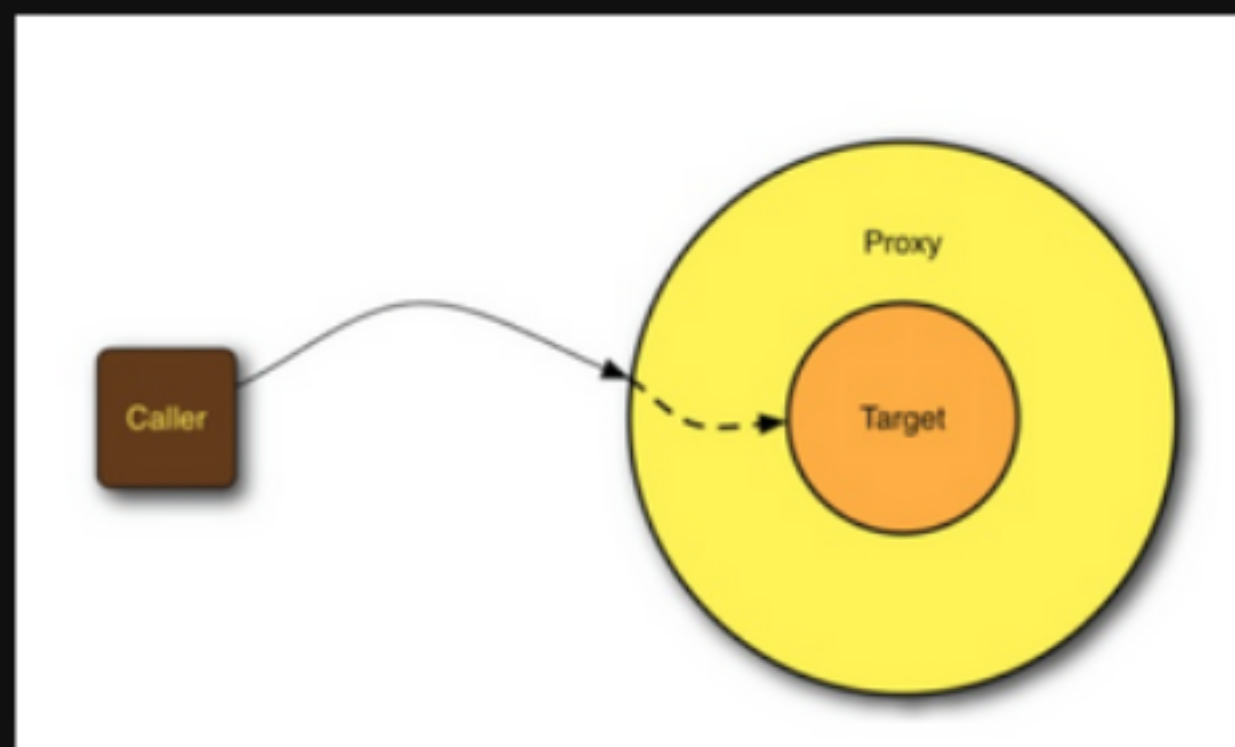
AOP Terminology



- A Joinpoint is a candidate point in the execution of the application where an aspect can be plugged in
- Advice is the actual implementation of our aspect
- A Pointcut defines at what joinpoints advice should be applied

AOP Weaving

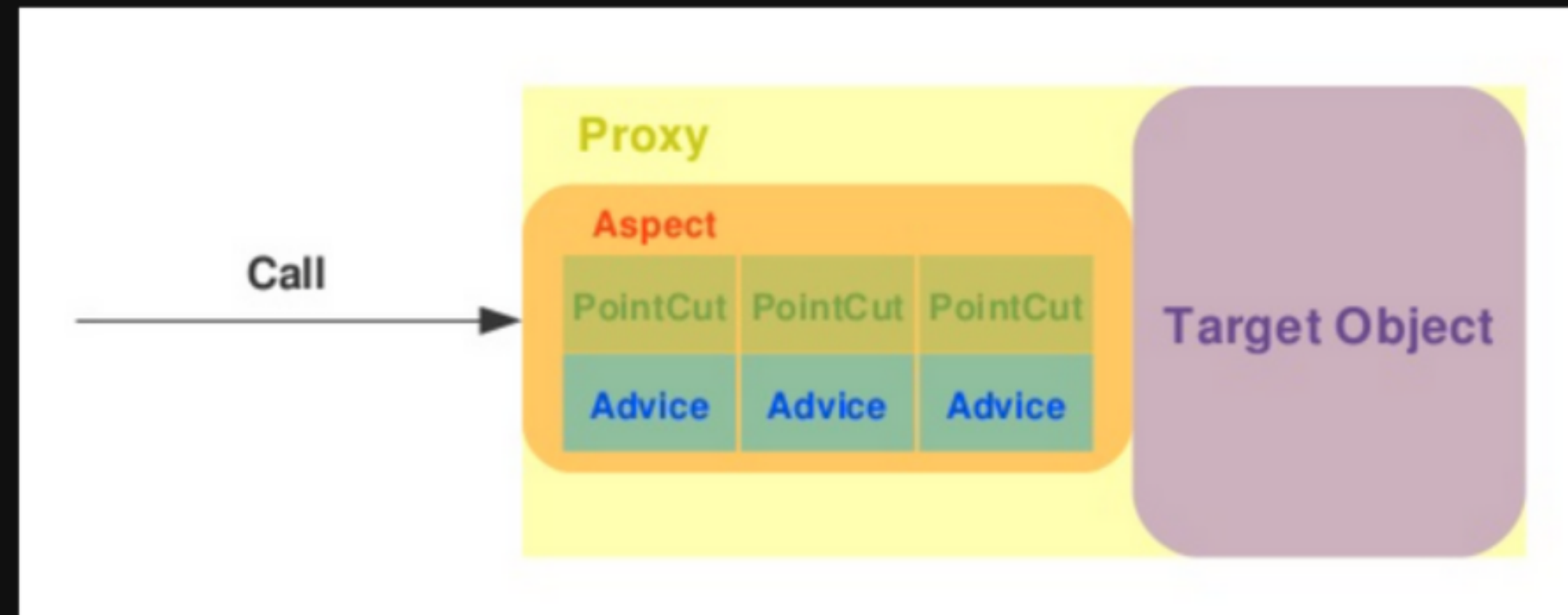
Weaving is the process of applying aspects to a target object to create a new, proxied object



Weaving can be done in different ways:

- Compile time—Aspects are woven in when the target class is compiled. This requires a special compiler
- Classload time—Aspects are woven in when the target class is loaded into the JVM. This requires a special ClassLoader
- Runtime—Aspects are woven in sometime during the execution of the application. Typically, an AOP container will dynamically generate a proxy class that will delegate to the target class while weaving in the aspects

AOP Mechanics



- An Aspect encapsulates Advices and Pointcuts
- Target Object is Advised by one or more Aspects
- The Proxy will manage how Aspects are applied to specific Pointcuts



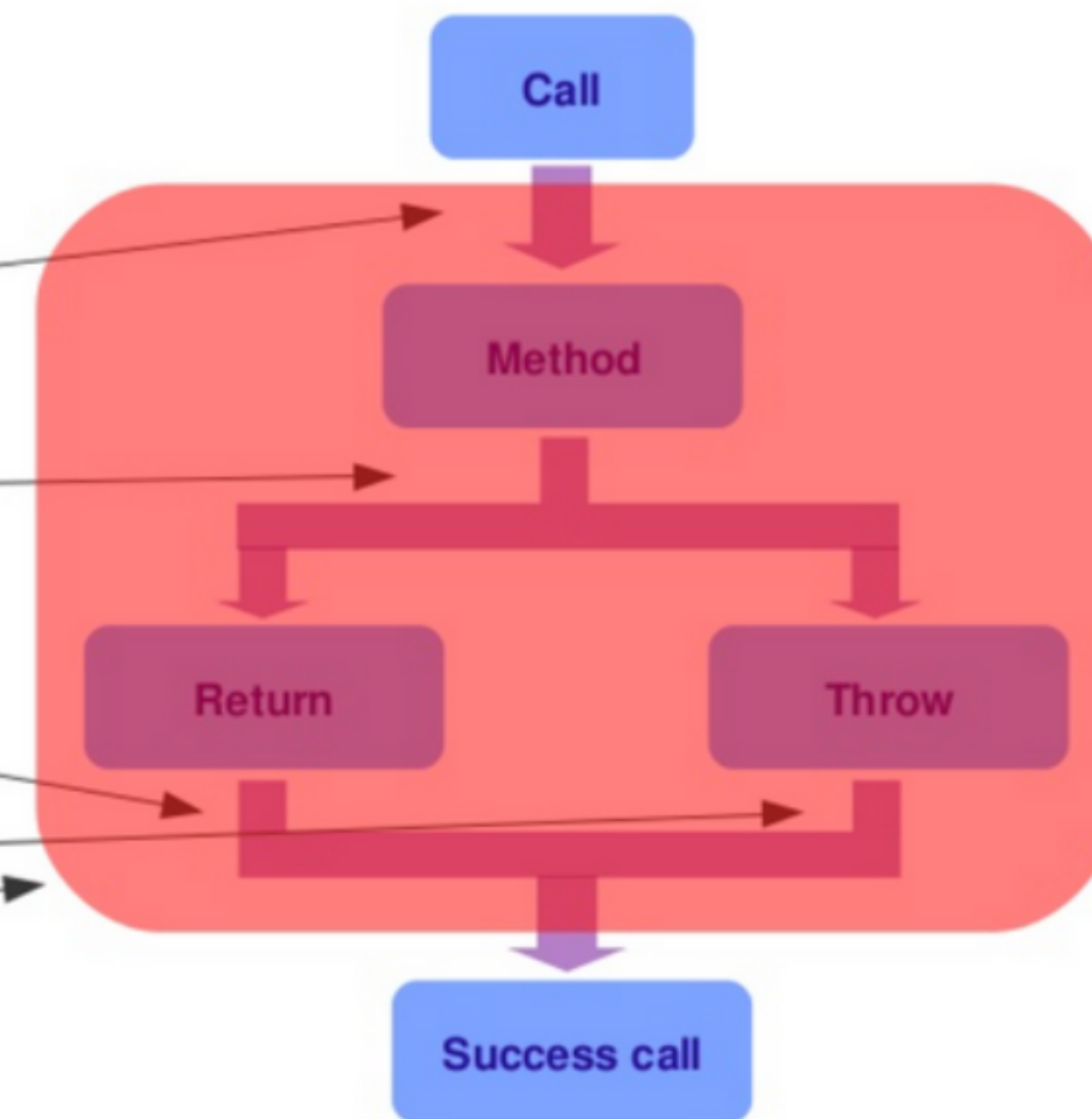
Spring AOP

- Implemented in pure Java
- No need for special compilation process
- Only Spring Beans can be advised
- Only method joinpoints are supported
- Uses runtime weaving

For more powerful AOP features, Spring supports AspectJ.

Advice Types

- Before
- After
- After Returning
- After Throwing
- Around



Before Advice

```
public class HijackBefore implements MethodBeforeAdvice {  
  
    @Override  
    public void before(Method method, Object[] args, Object target) throws Throwable {  
        System.out.println("Advised method will execute");  
    }  
}
```

After Returning Advice

```
public class HijackAfter implements AfterReturningAdvice {  
    @Override  
    public void afterReturning(Object returnValue, Method method,  
Object[] args, Object target) throws Throwable {  
        System.out.println("Advised method has executed");  
    }  
}
```

Metadata

```
<bean id="target" class="AdvisedBean"/>
<bean id="before" class="aop.HijackBefore"/>
<bean id="after" class="aop.HijackAfter"/>

<bean id="proxy" class="org.springframework.aop.framework.ProxyFactoryBean">
  <property name="target" ref="target"/>
  <property name="interceptorNames">
    <list>
      <value>before</value>
      <value>after</value>
    </list>
  </property>
</bean>
```

{ Hands On }

Spring AOP

Spring Transaction Management

Provides a consistent programming model across different transaction APIs such as Java Transaction API (JTA), JDBC, Hibernate, Java Persistence API (JPA), and Java Data Objects (JDO)

Transaction Manager

Create a Spring Hibernate Transaction Manager Bean

```
<bean id="transactionManager" class="org.springframework.orm.jpa.JpaTransactionManager"  
    <property name="entityManagerFactory" ref="emFactory"/>  
</bean>
```

Enable Spring AOP Transaction Management

```
<!-- Activate annotation-driven transactions -->  
<tx:annotation-driven transaction-manager="transactionManager"/>
```

Add Transactional Behaviour to Services

```
public class UserServiceImpl implements UserService {  
  
    // Other user service methods  
  
    @Transactional  
    @Override  
    public void addUser(User user) {  
  
        if (userDao.findByName(user.getUsername()) == null) {  
            userDao.save(user);  
        }  
  
    }  
  
}
```

If all methods require transactional behaviour, the whole class can be annotated with @Transactional

{ Exercise }

Add Spring Transactions to the JavaBank Application